Project 1 – Part 1: App Service vs. Azure Functions

Group: Martinho Pereira nº 66062, Wilker Martins nº 78013

Commit point:

1. Introduction

The base project is an auction backend (users, auctions, bids, questions, media) that runs as a Java web application on Azure App Service and stores its data in Cosmos DB. In this part we built a second version of the same service on Azure Functions (HTTP triggers) and compared the two on latency, throughput and cost. The motivation is the trade-off between an always-on, fixed-price server and a pay-per-use serverless platform that scales out automatically but may suffer cold starts.

Summary of findings. (i) The operations in cc.ops were reused unchanged; only three platform differences needed code. (ii) Under the same offered load, Functions was much faster than a single B1 App Service: median 63 ms vs 788 ms, p99 584 ms vs 9.0 s. It sustained about 27 responses/s vs 13, with no client timeouts (65 on App Service) and 128 of 130 sessions completed (65 on App Service). (iii) The cold start we measured was small, about 1 s. (iv) At low usage Functions is free, and it stays cheaper than App Service until roughly 14 million requests per month.

2. Design

Both versions run the same code from cc.ops, cc.data and cc.db against the same Cosmos DB database (400 RU/s) in the same region (Germany West Central). **App Service:** Linux plan B1 (1 core, 1.75 GB), Tomcat 10.1, Java 21, always on. **Functions:** Linux Consumption plan, Java 21, no pre-warmed instances.

In cc-proj-fun we added 22 HTTP-triggered functions (plus the provided ctrl/version), one per REST endpoint, in five classes (UsersFunctions, MediaFunctions, AuctionFunctions, BidFunctions, QuestionFunctions). Each function reads its parameters, calls the operation and turns the returned Result into a response. host.json sets the route prefix to rest, so paths are identical to App Service. A helper class (Http) replaces what JAX-RS did: JSON body parsing, the cc:session cookie, st/len pagination, and status, Set-Cookie and content-type handling. Three platform differences needed code:

1. **Session cookie.** The login cookie carries Max-Age, so java.net.CookieManager treats it as RFC 2965 and sends it back quoted. Jersey tolerated this, our first parser did not (every authenticated call returned 401). The function now strips the quotes.
2. **Route matching.** GET auction/search and GET auction/{id} have the same shape and the Functions host sent search to the by-id function, which now dispatches search itself.
3. **Binary bodies.** The Java worker delivers raw bytes only for Content-Type: application/octet-stream; for image/* it decodes the body as text and corrupts it (2000 random bytes arrived as 1891 characters). On Functions, images are therefore uploaded as octet-stream and the server detects the real type (JPEG/PNG/GIF/WebP) from the first bytes. The artillery scripts for Functions differ from the originals only in this header.

3. Evaluation

Methodology. artillery 2.x, run from the same machine and network for both versions, using workload1.yml: a 10 s warm-up plus 120 s at 1 new user/s, 130 virtual users in total, each running a mixed scenario (logins, listings, searches, media downloads, bids, questions). The database was populated once (100 users, 300 auctions, 1040 bids). The versions ran one after the other, never concurrently, since they share the database. Each version ran twice and we report the second run (steady state). Cold start was measured with curl after an idle period, and App

Service CPU with Azure Metrics. We report six representative endpoints: the heaviest reads (popular, recent), authentication, a write (bid), search and media.

Expectations. We expected App Service to be faster and steadier at low load, because it is always on and keeps the JVM, the Cosmos client and the connection pool warm, while Functions would pay a cold-start penalty on the first request after an idle period (the JVM start plus the Cosmos client creation). Under load we expected similar warm latency on both, with Functions possibly slightly worse from per-instance start-up and scale-out delay, and both bounded by the shared 400 RU/s database.

Results (ms, run 2).

Endpoint	App Service median / p95 / p99	Functions median / p95 / p99
GET /auction/any/popular	2417 / 9048 / 9801	105 / 550 / 804
GET /auction/any/recent	1720 / 7261 / 8868	86 / 284 / 460
POST /user/auth	1300 / 5945 / 7866	63 / 113 / 400
POST /auction/*/bid	1043 / 4965 / 6440	78 / 156 / 384
GET /auction/search	219 / 728 / 728	59 / 63 / 63
GET /media	340 / 2060 / 4404	55 / 89 / 302
All requests	**788 / 6312 / 9048**	**63 / 211 / 584**

Throughput (run 2, same offered load).

Metric	App Service	Functions
Responses received	2172	3631
Test duration (s)	about 167	about 134
Mean throughput (responses/s)	13	27
Peak throughput (10 s window, responses/s)	21	37
Client timeouts (10 s)	65	0
HTTP 429 (Cosmos DB)	0	2
Sessions completed (of 130)	65	128

The offered load was identical (130 users arriving over 130 s). Functions finished at about 134 s, so it kept up with the arrivals. App Service overran by about 35 s, because slow responses held sessions open and 65 sessions were cut by timeouts. The throughput difference therefore reflects how much work each version completed under the same load. It is not a measurement of maximum capacity for either. Both runs show HTTP 404 on GET /media (47 and 120), which comes from the workload requesting media ids that do not exist in some sessions, not from either server.

Cold start (GET /ctrl/version). On an idle Functions app the first request took 1.24 s and the next 0.19 s; App Service took 0.23 s. Since ctrl/version does not touch the database, this understates the real cold start, which also includes creating the Cosmos client.

Discussion. The expectation was met only for the first request after idle (about 1 s on Functions vs 0.23 s). Under load the result was the opposite of what we expected. The largest gaps are on the endpoints that do the most work inside the application on every request (popular, recent, auth), which fits CPU saturation: Azure Metrics showed the B1 CPU peaking at 100%, so a single core queued requests until many hit the 10 s timeout. We did not measure the number of Functions instances, so the explanation that the Consumption plan scaled out over several instances remains a hypothesis. Both versions share the 400 RU/s database; the two 429s on Functions suggest the faster version starts to reach that limit. **Limitations:** two runs per version, Functions ran first (database already warm for it), a single load level, and cold start measured only on a non-database endpoint.

4. Cost

List prices in USD, pay-as-you-go, checked in October 2026 (they vary by region and offer). **App Service Linux B1** (1 core, 1.75 GB): $13.14/month, charged whether or not it receives traffic; B2 (2 cores, 3.5 GB): $26.28/month. **Functions Consumption:** $0.20 per million executions plus $0.000016 per GB-s, with a monthly free grant of 1 million executions and 400,000 GB-s per subscription. The storage account and Application Insights are not counted, and the database is the same for both. Assumption for Functions: 0.5 GB billed memory and 150 ms billed time per request, i.e. 0.075 GB-s per request.

Scenario	App Service	Functions
Low usage: about 3 requests/min (about 130 000/month), long idle periods	$13.14/month, even when idle	$0, inside the free grant (0.13 M executions, about 9 700 GB-s)
High usage: 300 000 requests/day (9 M/month, about 3.5 requests/s on average)	$13.14 for one B1, but our test shows one B1 saturates (CPU 100%) at about 13 to 20 responses/s, so bursts need a B2 or two B1, about $26.28	about $6.00 (executions $1.60 + compute $4.40)

For N requests per month the Functions cost is about $1.4 \times 10^{-6} \times N - 6.6$ dollars. This equals the B1 price at $N \approx 14$ million (about 470 000 requests/day) and the B2 price at $N \approx 23.5$ million (about 780 000/day). Beyond that point a fixed-price instance is cheaper per request, provided it can handle the load. At low usage Functions costs nothing, and the price is the cold-start latency after idle periods.

5. Conclusion

We ported the backend to Azure Functions reusing the business logic unchanged, solved three platform-specific differences, and compared it with a B1 App Service on the same database. Functions gave much better latency and throughput under the same load, and lower cost at low and moderate volume, with a small cold-start penalty. With more time we would: record the instance count of Functions and the CPU of both versions during the runs to confirm why B1 saturated; test several load levels, including higher arrival rates, with the run order reversed; repeat the cold-start measurement several times on database endpoints; compare against a larger App Service tier (B2/P1v3) and against an always-ready Functions instance; and raise the Cosmos DB throughput to see which version then becomes the bottleneck.